



UNIVERSIDADE FEDERAL DO ABC
MCZA018 — Processamento Digital de Imagens
2026.1

Relatório Final

Sistema de Processamento Visual para Futebol de Robôs

Grupo 3

Igor Ladeia de Freitas	(RA 11201922180)
Gustavo Fernandes do Nascimento	(RA 11202021700)
Ryan Lucas da Silva	(RA 11202522362)
Eduardo Yukio Makita	(RA 11202020221)

25 de abril de 2026

Sumário

1	Introdução	2
2	Cenário de Aplicação	2
3	Fundamentação Teórica	2
3.1	Espaço de cores HSV	2
3.2	CLAHE (<i>Contrast Limited Adaptive Histogram Equalization</i>)	3
3.3	Operadores morfológicos	3
3.4	Segmentação por Watershed	3
3.5	Homografia	3
3.6	Filtro Gaussiano	3
4	Materiais e Métodos	4
4.1	Diagrama de Blocos Funcional do SPV	4
4.2	Descrição da Implementação	4
4.3	Lista de Arquivos	5
4.4	Análise Técnica	6
4.4.1	Métricas Objetivas	6
4.4.2	Observações Qualitativas	7
5	Laboratório Experimental	7
5.1	Roteiro do Laboratório Experimental	7
5.2	Análise dos Resultados Experimentais	8
5.2.1	Questões em Escala (1-5)	9
5.2.2	Questões Dissertativas	9
5.2.3	Síntese e Correlação com as Métricas Técnicas	10
6	Conclusões	11
	Referências Bibliográficas	11

1 Introdução

Este relatório descreve o desenvolvimento do **Sistema de Processamento Visual** (SPV) para futebol de robôs. O sistema utiliza exclusivamente técnicas clássicas de processamento digital de imagens, sem recorrer a métodos de aprendizado de máquina.

O objetivo principal é desenvolver um SPV que, a partir de uma câmera posicionada acima de um campo de futebol de robôs, seja capaz de:

1. Detectar e identificar individualmente cada robô pelo par ordenado de cores de sua *tag*;
2. Distinguir os dois times (azul e amarelo) pela cor central da *tag*;
3. Calcular a posição em centímetros e a orientação em graus de cada robô no campo;
4. Detectar a bola em jogo;
5. Operar em dois modos: vídeo pré-gravado e webcam ao vivo.

As seções seguintes apresentam o cenário de aplicação, a fundamentação teórica das técnicas empregadas, os materiais e métodos de implementação com análise técnica dos resultados, o laboratório experimental conduzido com usuário externo e as conclusões do projeto.

2 Cenário de Aplicação

O futebol de robôs consiste em partidas curtas disputadas em um campo reduzido entre robôs móveis autônomos. Cada robô é identificado por uma *tag* colorida fixada em sua parte superior: a cor central indica o time (azul ou amarelo) e duas cores laterais formam um par ordenado que identifica individualmente o robô dentro do time.

O desafio principal é que uma central de controle precisa saber, em tempo real, a posição, a orientação e a identidade de cada robô para tomar decisões táticas durante a partida. A abordagem padrão em ligas como a RoboCup Small Size League (SSL) é o uso de uma câmera aérea posicionada sobre o campo, que captura uma visão global do jogo [1].

A segmentação por cores é particularmente conveniente nesse contexto, pois o ambiente é controlado: a iluminação pode ser padronizada, as cores das *tags* são escolhidas para maximizar o contraste e o fundo do campo é uniforme. Essas condições favorecem o uso de técnicas clássicas de processamento de imagens baseadas no espaço de cores HSV.

3 Fundamentação Teórica

3.1 Espaço de cores HSV

O espaço de cores HSV (*Hue*, *Saturation*, *Value*) representa as cores em termos de matiz (H), saturação (S) e brilho (V). Diferentemente do espaço RGB, no qual variações de iluminação afetam simultaneamente os três canais, no HSV a informação cromática concentra-se no canal H, enquanto o brilho é isolado no canal V. Essa separação torna a segmentação por cor mais robusta a variações de iluminação, pois é possível definir faixas de matiz relativamente estáveis mesmo quando o brilho ambiente muda [2].

3.2 CLAHE (*Contrast Limited Adaptive Histogram Equalization*)

A equalização adaptativa de histograma com limite de contraste (CLAHE) é uma variante da equalização de histograma que opera localmente em sub-regiões da imagem. Cada sub-região tem seu histograma equalizado de forma independente, com um limite de contraste (*clip limit*) que impede a amplificação excessiva de ruído em regiões homogêneas. O resultado é uma imagem com contraste local aprimorado, o que melhora a separabilidade das cores antes da segmentação [3].

3.3 Operadores morfológicos

A abertura morfológica consiste na aplicação sequencial de uma erosão seguida de uma dilatação com o mesmo elemento estruturante. A erosão remove pequenos componentes e ruídos, enquanto a dilatação subsequente restaura o tamanho aproximado dos objetos remanescentes. Essa operação é amplamente utilizada como etapa de pós-processamento após a segmentação por limiarização, eliminando falsos positivos de pequena escala [2].

3.4 Segmentação por Watershed

O algoritmo de *Watershed* (divisor de águas) trata a imagem em níveis de cinza como uma superfície topográfica e simula uma inundação a partir de marcadores pré-definidos. A transformada de distância é aplicada à máscara binária para gerar um mapa de elevação, e componentes conexos dos picos dessa transformada servem como marcadores. A inundação prossegue até que bacias vizinhas se encontrem, definindo as linhas de separação. Essa técnica é útil para separar *blobs* próximos ou parcialmente sobrepostos [4].

3.5 Homografia

Uma homografia é uma transformação projetiva que mapeia pontos de um plano para outro por meio de uma matriz 3×3 . No contexto deste trabalho, a homografia é utilizada para converter coordenadas de pixel da imagem capturada pela câmera em coordenadas reais (em centímetros) sobre o plano do campo. A partir de quatro correspondências ponto-a-ponto entre os cantos do campo na imagem e suas posições reais conhecidas, a matriz de homografia é estimada e aplicada a todos os pontos detectados [5].

3.6 Filtro Gaussiano

O filtro Gaussiano realiza uma suavização da imagem por convolução com um *kernel* que segue a distribuição normal bidimensional. Essa operação reduz o ruído de alta frequência presente na imagem, atenuando variações espúrias de intensidade que poderiam gerar falsos positivos na etapa de segmentação. O tamanho do *kernel* controla o grau de suavização: *kernels* maiores produzem maior borramento [2].

4 Materiais e Métodos

4.1 Diagrama de Blocos Funcional do SPV

A Figura 1 apresenta o diagrama de blocos funcional do sistema de processamento visual. O pipeline é executado sequencialmente para cada frame capturado.

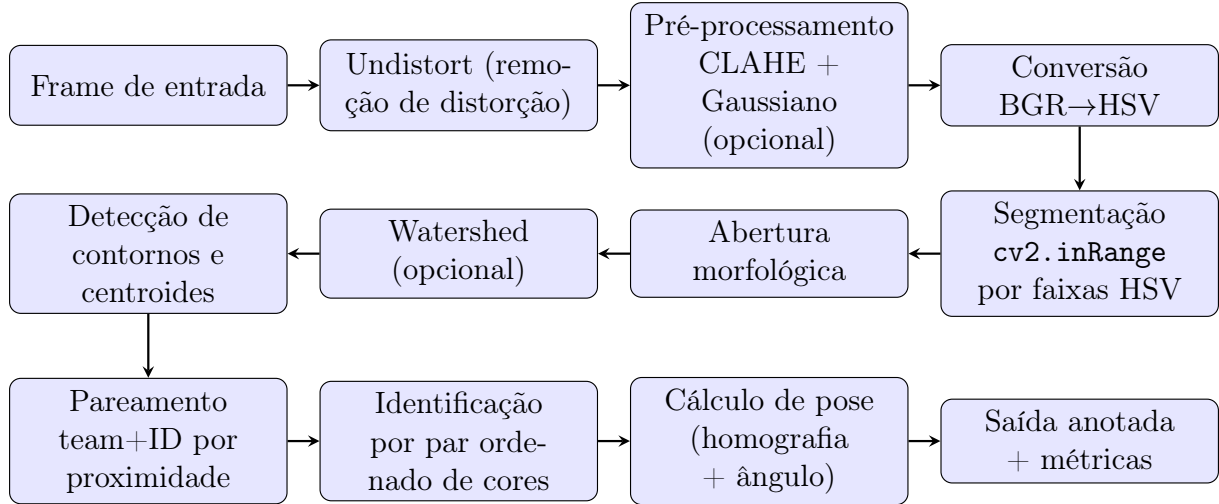


Figura 1: Diagrama de blocos funcional do SPV.

4.2 Descrição da Implementação

O sistema foi implementado em Python utilizando a biblioteca OpenCV [6]. Cada bloco do diagrama corresponde a um módulo ou função específica do pacote `robofoot_tracker`.

Remoção de distorção. O método `Tracker.undistort_frame` (em `robofoot_tracker/tracker.py`) aplica `cv2.undistort` quando a matriz de câmera e os coeficientes de distorção estão disponíveis na estrutura `CalibrationData` (calculados por `estimate_distortion`). Na ausência desses parâmetros, como no caso do vídeo de testes utilizado neste trabalho, a função atua como identidade, preservando o frame original.

Pré-processamento. A função `preprocess_frame` (em `robofoot_tracker/preprocessing.py`) converte o frame para HSV, aplica CLAHE com `clipLimit=1.5` e `gridSize=(8,8)` ao canal V (brilho), reconverte para BGR e finaliza com um filtro Gaussiano de `ksize=3` para suavização. Essa etapa é opcional (ativada por meio do parâmetro `preprocessing=True` na classe `Tracker`) e seu impacto quantitativo é analisado na Seção 4.4.1.

Conversão para HSV. A imagem pré-processada é convertida do espaço BGR para HSV via `cv2.cvtColor`, permitindo a segmentação por faixas de matiz.

Segmentação por cor. Para cada cor configurada (duas cores de time, quatro cores de ID e uma cor para a bola), a função `cv2.inRange` é aplicada com os limites HSV inferior e superior definidos na classe `ColorConfig` (em `robofoot_tracker/color_config.py`). O resultado é uma máscara binária por cor.

Abertura morfológica. Cada máscara binária passa por `cv2.morphologyEx(MORPH_OPEN)` com um elemento estruturante elíptico de 3×3 pixels, removendo pequenos componentes espúrios.

Watershed (opcional). Quando habilitado, o algoritmo de *Watershed* é aplicado apenas às máscaras de time. A transformada de distância gera um mapa de elevação, componentes conexos dos picos servem como marcadores e `cv2.watershed` separa *blobs* próximos. Essa etapa é implementada em `robofoot_tracker/detector.py`.

Deteção de contornos e centroides. Os contornos de cada máscara são extraídos e seus centroides calculados via momentos de imagem. Contornos com área abaixo de um limiar mínimo são descartados.

Pareamento team+ID por proximidade. Para cada *blob* de time detectado, os dois *blobs* de cor de ID mais próximos são associados por distância euclidiana. Essa lógica está implementada em `robofoot_tracker/detector.py`.

Identificação por par ordenado de cores. O vetor frontal do robô é definido como a direção do centroide do time para o ponto médio dos dois centroides de ID. O produto vetorial entre esse vetor frontal e o vetor do centroide do time para cada *blob* de ID determina qual cor está à esquerda e qual está à direita. O par ordenado (esquerda, direita) é consultado no dicionário `COLOR_PAIR_TO_ID` (em `robofoot_tracker/models.py`), que mapeia 10 pares válidos para os IDs de 0 a 9. Essa convenção por produto vetorial garante estabilidade do ID mesmo sob rotação completa do robô.

Cálculo de pose. A posição em centímetros é obtida aplicando a matriz de homografia (calculada em `robofoot_tracker/calibration.py`) ao centroide em pixels, mapeando diretamente coordenadas de imagem para coordenadas de campo. A orientação é calculada via `atan2` sobre o vetor frontal, com uma convenção de *offset* de 180° .

4.3 Lista de Arquivos

A estrutura principal do sistema é composta pelos seguintes arquivos:

`robofoot_tracker/` Pacote Python principal, com os seguintes módulos:

- `models.py` — *dataclasses* e dicionário `COLOR_PAIR_TO_ID`.
- `color_config.py` — faixas HSV das 7 cores (2 times + 4 IDs + bola).
- `preprocessing.py` — CLAHE + Gaussiano.
- `calibration.py` — homografia e UI de calibração interativa.
- `detector.py` — segmentação, detecção de *blobs* e pareamento.
- `geometry.py` — curvas de Bézier para bordas do campo.
- `viz.py` — anotação visual dos frames.
- `tracker.py` — API pública (classe `Tracker`).

`Trabalho_Final.ipynb` *Notebook* com o pipeline completo e exemplos.

`video.mp4` Vídeo de entrada usado nos experimentos.

video_annotated.mp4 Vídeo de saída anotado, gerado pelo sistema.

tag_technical_drawing.png, tags_colors_combinations.png Imagens auxiliares de referência dos *tags*.

4.4 Análise Técnica

4.4.1 Métricas Objetivas

O *tracker* foi executado sobre o vídeo `video.mp4` (7805 frames), com `expected_count=3` e `expected_ids=[3, 8, 9]`, em quatro configurações que combinam a ativação ou não do pré-processamento (CLAHE + Gaussiano) e do *Watershed*. A Tabela 1 apresenta os resultados.

Tabela 1: Métricas de desempenho do SPV em quatro configurações.

Métrica	Baseline	+Preproc.	+Watershed	+Ambos
FPS	326,20	228,08	225,35	175,84
Taxa de detecção de tag	97,41%	99,73%	97,32%	99,67%
Taxa de detecção da bola	98,50%	99,01%	98,50%	99,01%
Erro de contagem	12,31%	8,55%	12,39%	8,85%
Recall de ID	85,81%	88,94%	85,81%	88,93%
Precisão de ID	95,36%	96,54%	95,42%	96,83%
Jitter de orientação (°)	1,07	0,97	1,11	0,99
Tempo de processamento (s)	23,91	34,20	34,62	44,36

Três conclusões principais emergem dos resultados:

Impacto do pré-processamento. A ativação do CLAHE+Gaussiano produz o ganho mais expressivo: a taxa de detecção sobe de 97,41% para 99,73%, o *recall* de ID aumenta de 85,81% para 88,94% e o erro de contagem cai de 12,31% para 8,55%. A precisão de ID também melhora um pouco (95,36% → 96,54%). Esses ganhos evidenciam que a equalização adaptativa de histograma compensa variações locais de brilho e que a suavização Gaussiana reduz o ruído que causaria falsos negativos na segmentação por `cv2.inRange`. O custo é uma queda de FPS de 326,20 para 228,08 (cerca de 30%), ainda bem acima de qualquer requisito de tempo real. O jitter de orientação também apresenta leve redução (1,07° → 0,97°).

Impacto do Watershed. A ativação isolada do Watershed não produz ganho mensurável sobre a baseline (detecção 97,41% → 97,32%; *recall* quase idêntico), mas ocasiona em queda de FPS da mesma ordem que o pré-processamento (30%). O resultado sugere que, nas condições do vídeo testado, os *blobs* de time não chegam a se fundir com frequência suficiente para justificar a separação adicional. Esse é o motivo pelo qual o Watershed permanece desabilitado por padrão no sistema.

Combinação de ambos. A configuração *+Ambos* atinge o maior custo computacional (175,84 FPS, 44,36s de processamento), mas não supera substancialmente a configuração com apenas o pré-processamento ativo: a detecção cai ligeiramente (99,73% → 99,67%), o *recall* é praticamente idêntico e a precisão apresenta pequeno ganho (96,54% → 96,83%). Conclui-se que, para este cenário, o pré-processamento é a intervenção de maior custo-benefício.

4.4.2 Observações Qualitativas

As seguintes observações foram confirmadas durante o desenvolvimento e os testes do sistema:

- **Sensibilidade à iluminação:** o sistema é altamente sensível a variações de brilho ambiente. Mudanças bruscas de luz deslocam os valores HSV para fora das faixas calibradas, causando falhas de segmentação.
- **Recuperação pós-occlusão:** quando um robô é temporariamente coberto e depois descoberto, o ID é recuperado sem alterações, a identificação por par ordenado de cores é robusta a oclusões curtas.
- **Jitter de orientação:** medição quantitativa na Tabela 1 indica desvio-padrão circular médio de aproximadamente 1° (janelas de 15 frames com robôs estacionários, limiar de posição < 1 cm). O pré-processamento reduz o jitter em cerca de 10% (de $1,07^\circ$ para $0,97^\circ$).
- **Falsos positivos por fundo:** quando um time é habilitado mas não está presente em campo, objetos de fundo com cor próxima podem ser acidentalmente segmentados, em particular, o fundo branco do ambiente sendo detectado como amarelo. Com o time azul, há risco análogo com superfícies escuras.
- **Estabilidade sob rotação:** graças à convenção de ordenação por produto vetorial em torno do eixo frontal do robô, o ID permanece estável mesmo com rotação completa do robô, validando a correção feita sobre a convenção anterior, baseada em ângulo absoluto na imagem.
- **Deteção consistente da bola:** a bola é detectada de forma confiável em 98,5% dos frames, resultado do contraste favorável de sua cor laranja contra o fundo do campo.

5 Laboratório Experimental

5.1 Roteiro do Laboratório Experimental

O roteiro completo do Laboratório Experimental é um documento independente, submetido em separado junto aos entregáveis do projeto. O documento está organizado nas seguintes seções: Introdução, Procedimento Experimental, Questionário, e Consentimento e Registro em Vídeo. O caso de aplicação testado foi o futebol de robôs com operação ao vivo pela webcam por um usuário externo à equipe de desenvolvimento.

A Figura 2 reúne três registros do experimento: a tela de calibração do sistema (Fig. a), um frame anotado com as detecções (Fig. b) e a montagem física utilizada (Fig. c).



(a) Tela de calibração de cantos do campo.



(b) Frame anotado com detecções de robôs e bola.



(c) Montagem física do experimento com câmera aérea.

Figura 2: Registros do Laboratório Experimental.

5.2 Análise dos Resultados Experimentais

Sete usuários externos à equipe participaram do teste de campo em 17 de abril de 2026. Cada participante executou o experimento descrito no Roteiro do LEx e preencheu uma encuesta subjetiva de opinião contendo 11 questões em escala 1–5 e 6 questões dissertativas.

5.2.1 Questões em Escala (1-5)

A Tabela 2 apresenta a média e mediana das respostas por questão. Itens redigidos em sentido positivo (notas altas = opinião favorável) aparecem no topo; itens em sentido negativo (notas altas = opinião desfavorável) aparecem na parte inferior.

Tabela 2: Resultados da enquete subjetiva de opinião (N=7).

Questão	Média	Mediana
<i>Itens positivos (nota alta = opinião favorável)</i>		
Gostaria de usar esse sistema com frequência	4,83	5
Achei o sistema fácil de usar	4,86	5
Achei que várias funções foram bem integradas	5,00	5
Aprenderia a usar o sistema rapidamente	4,86	5
Me senti confiante ao usar o sistema	5,00	5
Achou o sistema interativo	4,86	5
<i>Itens negativos (nota alta = opinião desfavorável)</i>		
Achei o sistema desnecessariamente complexo	1,14	1
Precisaria de suporte técnico para usar o sistema	1,57	1
Achei que havia várias inconsistências	1,14	1
Achei o sistema complicado de usar	1,43	1
Precisei aprender muitas coisas antes de usar	1,71	1

Os resultados são consistentemente favoráveis: todos os itens positivos têm mediana 5 e média $\geq 4,83$, enquanto todos os itens negativos têm mediana 1 e média $\leq 1,71$. A maior dispersão aparece na questão “*Precisei aprender muitas coisas antes de usar*” (média 1,71), concentrada em dois respondentes que observaram complexidade na fase inicial de configuração, percepção coerente com a necessidade de calibração manual de cantos e cores.

5.2.2 Questões Dissertativas

As respostas abertas foram agrupadas por tema:

Aspectos mais elogiados (Q12). Aspectos relacionados à detecção em tempo real (posição, orientação ou análise dinâmica) foram mencionados por 5 dos 7 respondentes. Configuração e calibração de cores apareceram em 2 respostas. Destaques textuais: “*a forma que a posição do carrinho é identificada*” e “*a forma de identificar a direção*”.

Aspectos menos apreciados (Q13). Dois respondentes mencionaram *delay* da câmera ou *setup* lento; outros dois mencionaram complexidade do *setup* inicial. Três respondentes explicitamente afirmaram não ter pontos negativos a apontar.

Sugestões e comentários (Q14). Duas sugestões convergiram para a implementação de detecção da bola, funcionalidade que já consta no sistema, contudo, não havia representante disponível para simulação de bola durante o experimento. Outra sugestão foi o uso de fundo escuro, o que melhora o contraste das *tags* coloridas.

Objetivo percebido (Q15). Todos os 7 respondentes identificaram corretamente o objetivo do sistema (detecção de posição, ID, orientação dos robôs, monitoramento em competição), confirmando o atendimento ao critério “*entendeu a aplicação do sistema*” do questionário de compreensão.

Experimentos realizados (Q16). Os relatos descrevem predominantemente três atividades: reposicionamento dos robôs no campo, variação da quantidade de robôs e calibração de cores. As atividades relatadas coincidem com o procedimento previsto no Roteiro do LEx.

Resultados observados (Q17). Todos os 7 respondentes reportaram sucesso na detecção dos robôs. Um participante observou “*uma leve dificuldade se o ambiente for mais escuro*”, observação que coincide exatamente com a limitação de sensibilidade à iluminação documentada pela equipe na Seção 4.4.1 e na Seção 4.4.2.

5.2.3 Síntese e Correlação com as Métricas Técnicas

A enquete confirma, do ponto de vista do usuário externo, três pontos já observados pela equipe no desenvolvimento:

1. A taxa elevada de detecção medida objetivamente (97,4% a 99,7%, Tabela 1) é percebida pelo usuário como confiabilidade, todos reportaram resultados bem-sucedidos.
2. A sensibilidade à iluminação, identificada qualitativamente durante os testes de laboratório, foi independentemente notada por um usuário em condições de luz reduzida.
3. A complexidade da fase de calibração, conhecida internamente, emergiu como o único ponto consistente de atrito na experiência do usuário, motivando o ajuste automático de faixas HSV proposto como trabalho futuro.

A ausência de críticas relacionadas à identificação de robôs ou à estabilidade do ID (inclusive sob rotação) corrobora, qualitativamente, a robustez da convenção de ordenação por produto vetorial introduzida na implementação.

6 Conclusões

Atendimento dos objetivos

Todos os cinco objetivos estabelecidos na Introdução foram atingidos: o sistema detecta robôs e bola, identifica individualmente cada robô pelo par ordenado de cores, calcula posição em centímetros e orientação em graus, e opera nos dois modos previstos (vídeo pré-gravado e webcam ao vivo).

Pontos positivos

- Taxa de detecção de 97,4% na configuração *baseline* e 99,7% com pré-processamento ativado;
- Ganho quantitativo claro do pré-processamento (CLAHE+Gaussiano): +2,3 pontos percentuais na taxa de detecção e +3,1 no *recall* de ID, com FPS ainda acima de 220;
- Ordenação de IDs invariante a rotação, graças à convenção por produto vetorial;
- Interface simples de calibração interativa para definição dos cantos do campo;
- Suporte a métricas de acurácia quando valores esperados são fornecidos pelo usuário.

Limitações

- Alta sensibilidade à iluminação ambiente;
- Falsos positivos por fundo quando apenas um time está presente em campo;
- Parâmetros morfológicos e faixas HSV empiricamente calibrados, exigem re-calibração em novos ambientes;
- *Watershed* disponível, porém com custo de performance sem ganho de qualidade mensurável.

Trabalho futuro

Como trabalhos futuros, propõem-se: (i) o ajuste automático de faixas HSV por análise de histograma do frame; e (ii) melhoria no método de amostragem entre diferentes faixas de brilho reduzindo sensibilidade ao ambiente.

Referências Bibliográficas

- [1] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.
- [2] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4 edition, 2018.
- [3] Stephen M. Pizer, E. Philip Amburn, John D. Austin, Robert Cromartie, Ari Gese-lowitz, Trey Greer, Bart ter Haar Romeny, John B. Zimmerman, and Karel Zuiderveld. Adaptive histogram equalization and its variations. *Computer Vision, Graphics, and Image Processing*, 39(3):355–368, 1987.
- [4] Serge Beucher and Fernand Meyer. The morphological approach to segmentation: The watershed transformation. In *Mathematical Morphology in Image Processing*, pages 433–481. Marcel Dekker, 1993.
- [5] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2004.
- [6] OpenCV Team. OpenCV: Open Source Computer Vision Library. <https://opencv.org>, 2024.